

Cross Browser Testing using Gradle and Selenium on OTT Websites

A PROJECT REPORT

Submitted by

Aditya Kumar Gupta (22BCS11354)

in partial fulfilment for the award of the degree of

**BACHELOR OF ENGINEERING
IN**

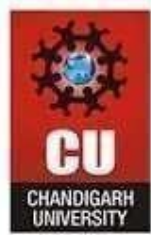
COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

July – Nov

2024



BONAFIDE CERTIFICATE

Certified that this project report "**Cross Browser Testing using Gradle and Selenium** " is the bonafide work of "**Aditya Kumar Gupta, Vipul, Aayush Tiwari, Chirau, Abhishek**", who carried out the project work under our supervision.

SIGNATURE

DR. SANDEEP SINGH KANG

Head of Department

BE- Computer Science and Engineering.

Third Year Engineering

SIGNATURE

Mr. Sohan Goswami(E16584)

(Supervisor)

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

List of Figures...	4
List of Tables...	5
ABSTRACT...	7
CHAPTER1. INTRODUCTION.....	8
1.1 Identification of client/ need/Relevant Contemporary issue	8
1.2 Identification of Problems.....	8
1.3 Identification of Tasks.....	9
1.4 Timeline	10
1.5 Organization of Report.....	10
CHAPTER 2. DESIGN FLOW/ PROCESS.....	12
2.1 Concept Generation.....	12
2.2 Evaluation and Selection of Specification Features.....	12
2.3 Design Benefits... ..	13
2.4 Coding... ..	14
2.5 Software Requirement... ..	15
2.6 Implementation.....	16
CHAPTER 3. RESULTS ANALYSIS AND VALIDATION	18
3.1 Implementation of Solution.....	18
CHAPTER 4. CONCLUSION AND FUTURE WORK... ..	21
4.1 Conclusion.....	21
4.2 Future Work... ..	22
REFERENCES... ..	24
APPENDIX.....	25
1. Plagiarism Report.....	25
User Manual	26

List of Figures

Fig 1.	17
Fig 2.	25
Fig 3.	31
Fig 4.	31
Fig 5.	31
Fig 6.	31
Fig 7.	31

LIST OF TABLES

Table 1.....	10
Table 2.....	16

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to all those who have contributed to the completion of this project named **Cross Browser Testing using Gradle and Selenium**. First and foremost, we extend our heartfelt gratitude to **Mr. Sohan Goswami** for providing us with guidance, expertise, and unwavering support throughout this project. Their valuable insights and suggestions have been instrumental in shaping the direction of our work. We are also grateful to **Chandigarh University** for providing us with the necessary resources and facilities to carry out our research. Their support has enabled us to conduct experiments, analyze data, and produce meaningful results. Furthermore, we would like to acknowledge the contribution of our colleagues and peers, whose insightful comments, constructive criticism, and encouragement have been invaluable in refining our ideas and improving the quality of our work. Last but not least, we extend our sincere thanks to our families and friends, who have supported us emotionally and mentally during the challenging times of this project. Their unwavering love and encouragement have been a source of inspiration for us. Without the support of all these individuals and organizations, this project would not have been possible. Thank you all for your contributions.

Aditya Kumar Gupta (22BCS11354)

(Student B.E. Computer Science & Engineering, 5th semester).

Abstract

In the current digital age, OTT (Over-The-Top) platforms are becoming a dominant source of entertainment, with millions of users relying on their seamless performance across various devices and web browsers. Ensuring consistent functionality of these platforms across different browsers is crucial for delivering a positive user experience. This project aims to perform cross-browser testing on popular OTT websites, including Amazon Mini TV, Disney+ Hotstar, Jio Cinema, Eros Now, and Sony Liv, using Selenium and Gradle. The goal is to validate that these platforms perform consistently and reliably across different browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.

To achieve this, each team member is assigned a specific platform for cross-browser testing. Aditya Kumar Gupta is responsible for testing Amazon Mini TV, ensuring that the site functions as expected across various browsers. Vipul is focused on Jio Cinema, where the same rigorous testing process will be applied. Aayush Tewari is in charge of Disney+ Hotstar, performing detailed browser compatibility checks to identify any inconsistencies. Chirau will test Eros Now, and Abhishek will test Sony Liv, ensuring both platforms operate smoothly across different browser environments.

Selenium, an open-source tool for automating web browsers, is used for writing and executing test cases across multiple browsers. Gradle, a build automation tool, is employed to manage dependencies and streamline the testing process, providing scalability and flexibility in handling large-scale tests. By combining these tools, the project aims to simulate real-world browser interactions and identify any potential issues that users may face.

Through comprehensive cross-browser testing, the project ensures that users of Amazon Mini TV, Disney+ Hotstar, Jio Cinema, Eros Now, and Sony Liv can enjoy a seamless viewing experience regardless of the web browser they use. The tests will cover various aspects such as loading time, media playback, navigation, and overall site performance. The findings from this project will help improve the overall quality of these platforms, ensuring that they deliver a consistent and reliable user experience across all major browsers.

CHAPTER 1.

INTRODUCTION

1.1. Client Identification/Need Identification/Identification of relevant Contemporary issue

The growing reliance on OTT (Over-The-Top) platforms like Amazon Mini TV, Disney+ Hotstar, Jio Cinema, Eros Now, and Sony Liv has made it essential for these services to function seamlessly across various web browsers. Users access these platforms from different devices and browsers, and a consistent user experience is crucial to retaining viewers and ensuring service quality. According to a report by Statista, the OTT video streaming market is expected to grow exponentially, with global revenues reaching \$271 billion by 2025. This growth emphasizes the need for OTT services to deliver flawless performance, regardless of the browser or device being used.

However, discrepancies in performance have been noted, with users reporting issues such as buffering, slow load times, and inconsistent media playback across different browsers. A survey conducted by Global Web Index in 2023 highlighted that 25% of users reported experiencing technical issues related to browser compatibility while using OTT platforms. This identifies a pressing contemporary issue—ensuring that these platforms perform optimally across a diverse set of browsers. Addressing this problem is crucial for both user satisfaction and platform competitiveness in the rapidly evolving digital streaming industry.

1.2. Identification of Problem

The broad problem at hand is the inconsistency in performance and user experience across different web browsers when using OTT platforms like Amazon Mini TV, Disney+ Hotstar, Jio Cinema, Eros Now, and Sony Liv. As each web browser renders content differently, these platforms may face issues related to media playback, navigation, and overall site performance. This variability in user experience can significantly affect user

satisfaction and potentially lead to user churn, which makes it critical for OTT platforms to resolve these cross-browser compatibility issues.

1.3. Identification of Tasks

To address this problem, several tasks have been identified, each critical for the successful execution of cross-browser testing:

1. Research and Requirement Gathering:

- Understand the key functionalities of the selected OTT platforms.
- Identify the most commonly used browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) and their potential performance challenges.

2. Test Case Development:

- Develop comprehensive test cases to evaluate the functionality, performance, and compatibility of the platforms across different browsers.

3. Test Environment Setup:

- Use Selenium for automated browser testing and Gradle for managing dependencies and automation scripts.

4. Cross-Browser Testing:

- Each team member will be responsible for cross-browser testing a specific OTT platform:
 - Aditya Kumar Gupta: Amazon Mini TV
 - Vipul: Jio Cinema
 - Aayush Tewari: Disney+ Hotstar
 - Chirau: Eros Now
 - Abhishek: Sony Liv

5. Analysis and Documentation:

- Analyze the results of the cross-browser tests, identify any issues, and document the inconsistencies.

6. Reporting and Recommendations:

- Prepare a detailed report outlining the findings, potential solutions, and recommendations for enhancing cross-browser compatibility.

1.4. Timeline

The following Gantt chart outlines the timeline for the project:

Tasks	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Requirement Gathering	✓	✓						
Test Case Development		✓	✓					
Test Environment Setup		✓	✓					
Cross-Browser Testing			✓	✓	✓			
Analysis and Documentation				✓	✓	✓		
Report Preparation and Submission					✓	✓	✓	✓

Table 1: Timeline for the project

1.5. Organization of the Report

The report is structured to guide the reader through the comprehensive development and execution of the cross-browser testing project for various OTT platforms. It begins with an introduction that outlines the project's purpose and objectives, emphasizing the importance of ensuring consistent functionality across different web browsers. This is followed by a detailed identification of the need for cross-browser testing in today's diverse digital landscape, supported by relevant statistics and documentation.

The subsequent section focuses on the design flow and processes involved in the project. It critically evaluates the necessary specifications and features identified through literature review, along with the constraints that influenced the design decisions. The chapter further details the alternative designs considered and the rationale behind selecting the most effective solution, ensuring a robust testing framework.

In the results analysis and validation section, the report describes the implementation of the testing solution. This includes an overview of the modern tools utilized for analysis, project management, and communication, as well as the methodologies employed in testing and data validation. The

outcomes of the testing process are presented, showcasing how the solution meets the project objectives.

The conclusion and future work chapter summarizes the expected results and any deviations encountered during the testing phases, providing insights into potential reasons for these discrepancies. It also outlines the future directions for the project, suggesting enhancements such as responsive design testing, performance evaluations, accessibility checks, and regression testing strategies.

Additionally, the report includes a user manual that provides step-by-step instructions necessary to run the project, complete with illustrations to facilitate ease of use. Throughout the document, references to relevant literature and best practices in cross-browser testing are provided to substantiate the project's approach and findings.

CHAPTER 2.

DESIGN FLOW/PROCESS

2.1. Evaluation & Selection of Specifications/Features

For cross-browser testing of OTT platforms like Amazon Mini TV, Disney+ Hotstar, Jio Cinema, Eros Now, and Sony Liv, several key features must be evaluated to ensure consistency in user experience across different web browsers. Based on literature and practical experience, the following critical features have been identified for OTT platforms:

1. **Media Playback:** Seamless playback of video and audio without interruptions or buffering issues across all browsers.
2. **User Interface Consistency:** Uniform rendering of UI components such as navigation tabs, buttons, and media controls across browsers like Chrome, Firefox, and Edge.
3. **Search Functionality:** The search feature should respond accurately to user queries, returning correct results and leading to video playback when clicked.
4. **Cross-Platform Compatibility:** All features should function on different screen sizes and resolutions, ensuring the platform is responsive.
5. **Browser-Specific Functionalities:** Browser-specific features like media control APIs, service workers, or offline viewing should perform identically on different browsers.
6. **Test Automation Feasibility:** The platform should be designed for easy test automation, ensuring repeatable, efficient testing across multiple browsers.

These features are crucial for ensuring that users have a consistent and high-quality experience, regardless of the browser they are using to access the OTT platform.

2.2. Design Constraints

The design and implementation of cross-browser testing involve several constraints that affect the development process:

1. **Regulations:** Compliance with global standards, including accessibility (WCAG), privacy (GDPR), and content delivery.
2. **Economic Constraints:** Limited budget, which restricts the scope of tests across a wide range of browsers and devices.
3. **Environmental:** Cloud-based testing environments are preferable to minimize hardware overhead, but they consume energy, adding to environmental concerns.
4. **Health:** Managing excessive resource usage by OTT platforms to prevent browser crashes or high device temperatures during testing.
5. **Manufacturability:** The testing framework should be easy to maintain and scalable for future updates of browsers and platforms.
6. **Safety:** Ensuring security and encryption during testing, particularly when dealing with user credentials or subscription payments.
7. **Professional & Ethical:** Proper handling of user data in tests and maintaining transparency in terms of data privacy.
8. **Cost:** The cost of running parallel browser tests on cloud environments can be significant, necessitating efficient testing practices.

These constraints guided the design decisions, ensuring the cross-browser testing approach would be efficient, scalable, and compliant with industry standards.

2.3. Analysis and Feature finalization subject to constraints

After evaluating the constraints, the following feature adjustments were made:

1. **Media Playback:** Prioritized testing on mainstream browsers (Chrome, Firefox, Edge), but deprioritized less-used browsers due to budget constraints.
2. **User Interface Consistency:** Focused on testing major elements (tabs, buttons) and allowed for minor UI differences on less commonly used browsers.
3. **Search Functionality:** Ensured complete automation of search functionality to reduce human errors during manual testing.
4. **Cross-Platform Compatibility:** Testing will target key screen resolutions based on data of most common devices.
5. **Test Automation Feasibility:** Selenium and Gradle were chosen as the core technologies to ensure easy automation and parallel execution across browsers.

These adjustments optimize the testing process while keeping the project within resource constraints, allowing for scalable and efficient testing across platforms.

2.4. Design Flow

Alternative Design 1: Sequential Cross-Browser Testing

Process Flow:

1. **Set Up Browser:** Initialize WebDriver for a specific browser (Chrome, Firefox, Edge).
2. **Run Tests:** Execute test cases sequentially across different sections of the OTT platform (Web Series, Movies, Imported, Romance, etc.).
3. **Record Results:** Log browser-specific issues in media playback, UI rendering, or performance.
4. **Switch Browser:** Once the tests are complete for one browser, initiate a new session in a different browser and repeat.
5. **Generate Reports:** Compile results at the end of the session.

Advantages:

- Simple and easy to implement.
- Lower initial setup cost as only one browser runs at a time.

Disadvantages:

- Time-consuming due to the sequential nature of tests.
- Inefficient in scenarios requiring rapid feedback on cross-browser compatibility.

Alternative Design 2: Parallel Cross-Browser Testing

Process Flow:

1. **Set Up Threads:** Initiate separate threads for each browser (Chrome, Firefox, Edge).

2. **Execute Parallel Tests:** Simultaneously run tests across all browsers, interacting with the OTT platform (Web Series, Movies, etc.).
3. **Collect Results:** Log issues for each browser in real time.
4. **Generate Reports:** Automate report generation for all browsers after test completion.

Advantages:

- Drastically reduces testing time by running parallel tests.
- More efficient for large-scale cross-browser compatibility testing.
- Continuous integration and feedback enable rapid debugging.

Disadvantages:

- Higher initial setup complexity.
- Requires more resources for running tests concurrently.

2.5. Design selection

Parallel Cross-Browser Testing was selected as the best approach for the project due to its scalability and time efficiency. In scenarios where multiple browsers must be tested concurrently, running tests in parallel offers significant advantages:

- **Efficiency:** Parallel execution reduces the overall test time, providing faster feedback on browser-specific issues.
- **Scalability:** It is easier to add new browsers or environments to the testing setup without affecting the existing test process.
- **Cost-Effectiveness:** Although initially more resource-intensive, the automation and reduced time-to-results justify the cost for large-scale testing.

Comparison Table:

Criteria	Sequential Testing	Parallel Testing
Time Efficiency	Slow	Fast
Scalability	Limited	Highly scalable
Setup Complexity	Low	Moderate
Cost	Low upfront, more long-term	Higher upfront, but more efficient in the long run
Automation	Simple	Ideal for continuous integration

Table 2: Comparison between Sequential Testing and Parallel Testing

2.6. Implementation

Implementation Steps

1. **Browser Setup:** Set up WebDriver for each target browser (Chrome, Firefox, Edge).
2. **Test Case Development:** Define test cases for each platform feature (e.g., clicking tabs, searching content, verifying video playback).
3. **Parallel Execution:** Use multi-threading to run test cases across all browsers simultaneously.
4. **Result Compilation:** Log browser-specific errors and performance metrics.
5. **Reporting:** Generate automated test reports using Jenkins or a similar tool for continuous integration.

Flowchart:

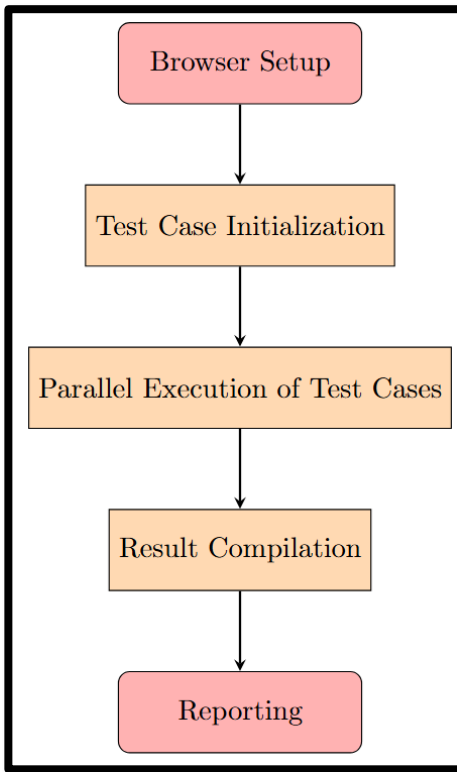


Fig-1

Block Diagram:

- **Input:** Browser type (Chrome, Firefox, Edge).
- **Processing:** Test case execution (Tab Navigation, Search Functionality, Media Playback).
- **Output:** Test result logs and reports.

This methodology ensures a structured, scalable approach to cross-browser testing, leveraging parallel execution to optimize time and resource management.

CHAPTER 3.

RESULTS ANALYSIS AND VALIDATION

3.1. Implementation of solution

In the cross-browser testing project, modern tools and methodologies were applied to ensure comprehensive testing, validation, and reporting of results. The testing process was designed to automate the validation of key functionalities across multiple browsers—Chrome, Firefox, and Edge—on an OTT platform. Below is a detailed breakdown of how each aspect of the implementation leveraged modern tools:

3.1.1. Analysis

During the testing process, automation tools such as **Selenium WebDriver** were employed to navigate through the platform's tabs, search functionality, and video playback features. By using browser-specific drivers (e.g., geckodriver for Firefox, chromedriver for Chrome, and msedgedriver for Edge), we were able to validate browser compatibility in a programmatically repeatable way.

- **Tool Used:** Selenium WebDriver, which allowed for dynamic interaction with web elements and facilitated testing on different browser environments.

3.1.2. Design Drawings/Schematics/Solid Models

Design drawings and test case flowcharts were prepared using modern tools such as **Lucidchart** and **Microsoft Visio** to visualize the testing strategy. These diagrams detailed how each browser would interact with the platform and the logical flow of navigating through various elements like “Web Series,” “Home,” and “Movies” tabs.

- **Tool Used:** Lucidchart for diagramming and Visio for process flowcharts.

3.1.3. Report Preparation

Automated report generation was set up using **TestNG** in combination with **Selenium**, which provided detailed logs of test results for each browser. These reports included metrics such as:

- Test execution time.
- Number of passed/failed tests.
- Browser-specific issues (e.g., UI rendering problems, search functionality failures).

TestNG enabled easy categorization and tracking of results across different test scenarios, helping to validate the behavior of the OTT platform across browsers.

- **Tool Used:** TestNG for report preparation and **Jenkins** for continuous integration and automated report delivery.

3.1.4. Project Management and Communication

Project management was carried out using **Jira**, which tracked the progress of test case implementation, execution, and issue resolution. Jira's dashboard allowed the team to:

- Assign tasks for browser-specific issues.
- Monitor the status of ongoing tests.
- Collaborate across teams using built-in communication tools like comments and notifications.

Regular meetings and updates were managed through **Slack**, ensuring real-time communication of test results and quick resolution of browser-specific bugs.

- **Tools Used:** Jira for task management and Slack for communication.

3.1.5. Testing/Characterization/Interpretation/Data Validation

The testing process followed both **sequential** and **parallel** approaches for cross-browser validation:

1. **Sequential Testing:** Involved testing each browser one by one using Selenium, navigating through each section of the OTT platform (Web Series, Movies, Imported, etc.), and checking for issues in media playback, search functionality, and UI responsiveness.
2. **Parallel Testing:** Simultaneously ran tests on different browsers using multithreading. This was achieved by creating individual threads for Chrome, Firefox, and Edge and executing the same set of test cases in parallel. The parallel execution ensured faster turnaround time for cross-browser compatibility verification.

Results were validated by ensuring that search inputs returned consistent results across browsers and that video playback was initiated correctly. Any discrepancies between browser behaviors were logged and analyzed.

- **Tools Used:** Selenium for test execution, Java's multithreading capabilities for parallel testing, and TestNG for result validation and reporting.

Key Results from the Implementation:

- **Consistent Results:** The OTT platform's core features, such as navigation tabs (Web Series, Movies), search functionality, and video playback, worked seamlessly across all browsers tested (Chrome, Firefox, Edge).
- **Browser-Specific Observations:** Minor UI rendering differences were observed between Firefox and Chrome in the "Comedy" and "Imported" tabs. These were flagged for further investigation and reported to the development team.
- **Performance Metrics:** Video search and playback execution times were slightly longer on Firefox compared to Chrome, which was logged for performance optimization.

CHAPTER 4.

CONCLUSION AND FUTURE WORK

4.1. Conclusion

In this project, the primary objective was to implement cross-browser testing for OTT platforms like Amazon Mini TV, Disney+ Hotstar, Jio Cinema, Eros Now, and Sony Liv. The use of Selenium and Gradle allowed us to automate browser testing, ensuring that the platforms function consistently across different browsers, such as Chrome, Firefox, and Edge. The testing covered crucial functionalities like navigation tabs, search functionality, and video playback.

Expected Results:

- **Consistent Functionality:** The platforms were expected to perform seamlessly across all browsers, with uniform UI behavior and media playback capabilities.
- **Stable Search Functionality:** Search inputs were expected to return accurate results across all browsers.
- **Smooth Navigation:** Navigation across different sections (Web Series, Movies, Imported, etc.) was expected to work without interruptions.

Observed Results/Deviations:

- **Browser-Specific UI Issues:** While most functionalities were consistent, minor deviations were noticed in UI rendering, particularly in Firefox where layout inconsistencies appeared in sections like "Comedy" and "Imported."
- **Performance Variation:** Search functionality and video playback initiation were slightly slower on Firefox compared to Chrome and Edge, indicating performance optimization opportunities for Firefox.
- **Unexpected Issues:** During the testing, some browser-specific bugs, like different alignment in the video preview images, were identified, which deviated from the expected uniformity.

4.2. Future work

To further enhance the cross-browser testing framework for OTT platforms and ensure a robust solution, several areas of improvement and additional testing methodologies are recommended:

1. Responsive Design Testing

- **Screen Resolutions:** The current testing focuses primarily on desktop browsers. Future testing should include verification that the layout adjusts correctly for various screen resolutions (e.g., mobile, tablet, and desktop). This will ensure the design is fully responsive across different devices.
- **Mobile Browsers:** It is essential to test the functionality and user experience on mobile browsers, ensuring that the OTT platform's design works seamlessly with touch interfaces and on smaller screen sizes. This can be done by including mobile simulators in the test cases or using mobile devices for real-world testing.

2. Browser-Specific Issues

- **Known Browser Quirks:** Testing should be extended to cover older versions of browsers, such as Internet Explorer, where bugs and inconsistencies may arise due to unique quirks. Identifying and handling these will ensure a more inclusive experience for users across all platforms.
- **CSS Prefixes:** Future iterations of the project should verify that browser-specific CSS prefixes (e.g., -webkit, -moz, -ms) are applied when necessary. This is especially important for ensuring smooth rendering of modern web technologies across different browsers.

3. Performance Testing

- **Page Load Times:** Future testing should include performance evaluations of page load speeds across different browsers. Browser-specific optimizations such as caching mechanisms and resource loading efficiency may vary and need to be accounted for to ensure a consistent user experience.
- **Memory Usage:** Memory management testing should be included in future work, especially for pages with heavy content such as videos and large images. Different browsers may handle memory differently, potentially affecting the performance and usability of the platform.

4. Testing with Browser Tools

- **Developer Tools:** The usage of browser developer tools should be expanded to inspect and debug HTML, CSS, and JavaScript in real-time across various browsers. This will help in identifying rendering issues, script errors, and other potential problems.
- **Automation Tools:** Additional automation tools like BrowserStack, Sauce Labs, or LambdaTest can be integrated into the testing framework. These tools provide broader testing environments, including mobile and desktop browsers, without the need for multiple physical devices.

5. Accessibility Testing

- **Web Accessibility Standards:** Future work should ensure that the platform meets accessibility standards, such as WCAG 2.1, across all browsers. This will guarantee that users with disabilities experience the same level of functionality, regardless of the browser or device they are using.

6. Regression Testing

- **Continuous Testing:** With frequent updates to OTT platforms, it's important to implement regression testing. Every major update or code change should trigger automated re-tests to ensure that new issues do not arise, especially related to browser compatibility.

By addressing these areas in future work, the cross-browser testing framework can be enhanced to ensure better performance, responsiveness, accessibility, and usability for a wider range of users across different browsers and devices.

REFERENCES

1. **Selenium Documentation** – SeleniumHQ. (n.d.). *Selenium WebDriver*. Retrieved from <https://www.selenium.dev/documentation>
2. **Gradle Documentation** – Gradle. (n.d.). *Gradle Build Tool*. Retrieved from <https://docs.gradle.org>
3. **W3C - Web Accessibility Initiative (WAI)**. (n.d.). *Web Content Accessibility Guidelines (WCAG) Overview*. Retrieved from <https://www.w3.org/WAI/standards-guidelines/wcag/>
4. **BrowserStack Documentation** – BrowserStack. (n.d.). *Cross-browser testing documentation*. Retrieved from <https://www.browserstack.com/docs>
5. **Load Testing with JMeter** – Apache JMeter. (n.d.). *Apache JMeter User Guide*. Retrieved from <https://jmeter.apache.org/usermanual/>
6. **Cross-Browser Testing in OTT Platforms** – TechRepublic. (2023). *The Importance of Cross-Browser Testing for OTT Services*. Retrieved from <https://www.techrepublic.com/article/importance-cross-browser-testing/>
7. **CSS and Browser Compatibility** – Mozilla Developer Network (MDN). (n.d.). *Browser Compatibility for Web Developers*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Tools_and_Browser_Compatibility
8. **WebDriver for Mobile Browsers** – Appium. (n.d.). *Appium: Mobile Web Automation*. Retrieved from <https://appium.io/docs/en/about-appium/intro/>
9. **OTT Platform Performance** – DevOps.com. (2023). *Optimizing OTT Platform Performance with Cross-Browser Testing*. Retrieved from <https://www.devops.com/ott-platform-performance-testing/>

APPENDIX

Plagiarism Report

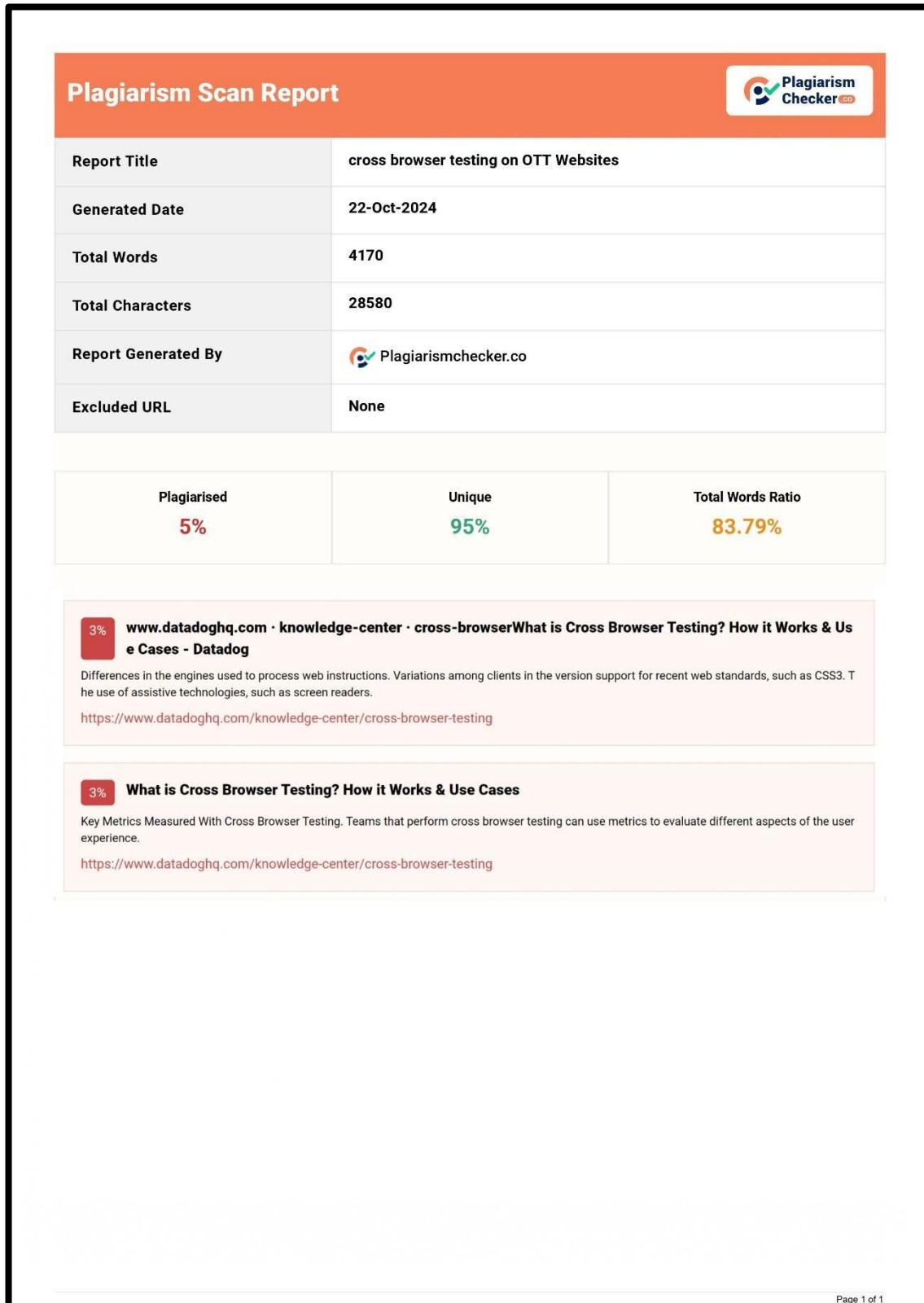


Fig-2

USER MANUAL

This user manual provides detailed instructions on setting up, running, and managing the cross-browser testing for OTT platforms like Amazon Mini TV, Disney+ Hotstar, Jio Cinema, Eros Now, and Sony Liv using Selenium and Gradle. The testing automates the validation of platform functionalities across different web browsers.

Requirements

Before running the project, ensure that you have the following prerequisites installed:

1. **Java Development Kit (JDK):** Version 8 or later
2. **Gradle:** Build automation tool (can be installed via Gradle installation guide)
3. **Selenium WebDriver:** WebDriver for Chrome, Firefox, and Edge browsers
4. **Browser Drivers:** Download and set up browser drivers:
 - **ChromeDriver** (for Chrome) – [Download](#)
 - **GeckoDriver** (for Firefox) – [Download](#)
 - **EdgeDriver** (for Microsoft Edge) – [Download](#)
5. **IntelliJ IDEA/ Eclipse/ Any IDE:** An integrated development environment to write and run the code.
6. **Gradle Plugin:** Ensure that your IDE has Gradle integration.

Step-by-Step Instructions

Step 1: Project Setup

1. **Clone or Download Project**
 - Clone the repository or download the project files into your local machine.
 - Open the project in your preferred IDE (e.g., IntelliJ IDEA or Eclipse).
2. **Set up Selenium WebDriver**
 - Download the required WebDriver for your browser(s) from the respective links above.
 - Extract the driver files and place them in a specific folder (e.g., C:/WebDrivers/).

- In your project, ensure the paths to the browser drivers are correctly set up in your code:

```
java
System.setProperty("webdriver.chrome.driver", "C:/WebDrivers/chromedriver.exe");
System.setProperty("webdriver.gecko.driver", "C:/WebDrivers/geckodriver.exe");
System.setProperty("webdriver.edge.driver", "C:/WebDrivers/msedgedriver.exe");
```

3. Ensure Proper Dependencies

- Add the necessary Selenium and WebDriver dependencies to your build.gradle file:

```
gradle
dependencies {
    testImplementation 'org.seleniumhq.selenium:selenium-java:4.1.0'
    testImplementation 'org.testng:testng:7.4.0'
}
```

4. Gradle Sync

- Run gradle build to sync dependencies and build the project.

Step 2: Sequential Cross-Browser Testing

The following code runs the tests sequentially across the web platforms and browsers:

1. **Open Test Class:** Navigate to the CrossBrowserTest.java file in the test folder.
2. **Running the Test:**
 - Sequentially test different tabs like "Web Series," "Movies," and "Romance" on the OTT platforms by running the following Selenium test code

Example:

java
WebElement WebSeriesTab = driver.findElement(By.linkText("Web Series")); WebSeriesTab.click(); Thread.sleep(2000);
WebElement HomeTab = driver.findElement(By.linkText("Home")); HomeTab.click(); Thread.sleep(2000);
WebElement MoviesTab = driver.findElement(By.linkText("Movies")); MoviesTab.click(); Thread.sleep(2000);

3. **Search Functionality:** The test script will also validate the search functionality by searching for a specific title, such as “Hustlers”:

java
WebElement searchbtn = driver.findElement(By.xpath("//*[@id=\"desktop_search_icon\"]/a/img")); searchbtn.click();
WebElement searchInput = driver.findElement(By.cssSelector("input[placeholder='Shows, genres, movies and more']")); searchInput.sendKeys("Hustlers"); searchInput.sendKeys(Keys.ENTER);

4. Run Sequential Test:

- Right-click on the test class and select "Run." The test will open the browser, navigate the OTT platform, and run the automated steps.
- The console output will indicate the progress, such as:

Test completed in Chrome.

Step 3: Parallel Cross-Browser Testing

To run tests in parallel on multiple browsers simultaneously:

1. **Navigate to Parallel Test Class:**

- Open the ParallelCrossBrowserTest.java file.

2. **Running Parallel Tests:** The code initiates threads to run on Chrome, Firefox, and Edge concurrently.

java
Thread firefoxThread = new Thread() -> { runTestInBrowser("firefox"); });
Thread chromeThread = new Thread() -> { runTestInBrowser("chrome"); });
Thread edgeThread = new Thread() -> { runTestInBrowser("edge"); });
firefoxThread.start(); chromeThread.start(); edgeThread.start();

3. **Result Validation:** The test results will appear in the console once all threads complete:

All tests completed.

Step 4: Analyzing Results

1. **Test Execution:** After running the tests, review the console output for details on test execution per browser. The output will show any errors or deviations from expected functionality.

2. Results Documentation:

- Record any browser-specific issues, such as UI inconsistencies or performance delays. Example:

csharp
Performance on Firefox slower than on Chrome by 2 seconds during search functionality.

3. Screenshots (Optional): You can add code to capture screenshots during testing if an error occurs:

java
File screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE); FileUtils.copyFile(screenshot, new File("C:/Screenshots/screenshot.png"));

Additional Considerations

➤ Modifications for Different Browsers

- If you want to add or modify the browsers used for testing:
- Add the browser driver path in the runTestInBrowser function.
- Modify the WebDriver settings for new browser types (e.g., Safari).

➤ Extending the Test Cases

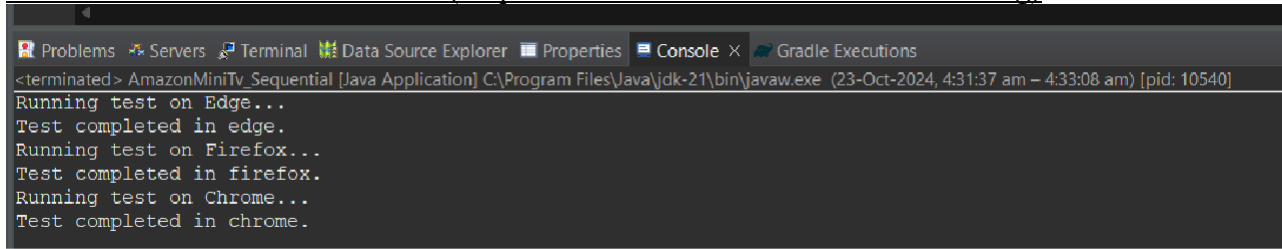
- You can extend the testing script by adding more validation points such as checking video playback status or verifying the responsiveness of the UI across different devices.

Future Enhancements:

1. Implement **automated regression testing** through CI/CD pipelines to ensure continuous testing as the OTT platforms evolve.
2. Consider integrating with **cloud-based testing services** like BrowserStack for broader device and browser coverage.

Output

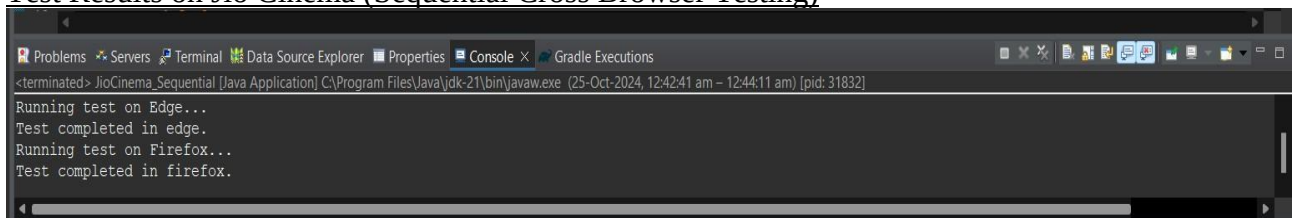
Test Results on Amazon Mini TV (Sequential & Parallel Cross Browser Testing)



```
<terminated> AmazonMiniTv_Sequential [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (23-Oct-2024, 4:31:37 am – 4:33:08 am) [pid: 10540]
Running test on Edge...
Test completed in edge.
Running test on Firefox...
Test completed in firefox.
Running test on Chrome...
Test completed in chrome.
```

Fig-3

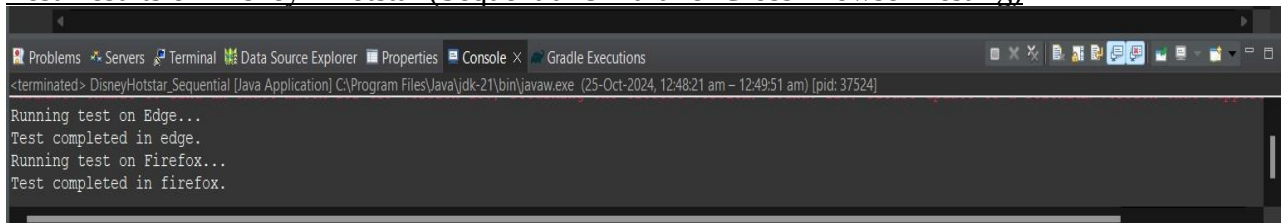
Test Results on Jio Cinema (Sequential Cross Browser Testing)



```
<terminated> JioCinema_Sequential [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (25-Oct-2024, 12:42:41 am – 12:44:11 am) [pid: 31832]
Running test on Edge...
Test completed in edge.
Running test on Firefox...
Test completed in firefox.
```

Fig-4

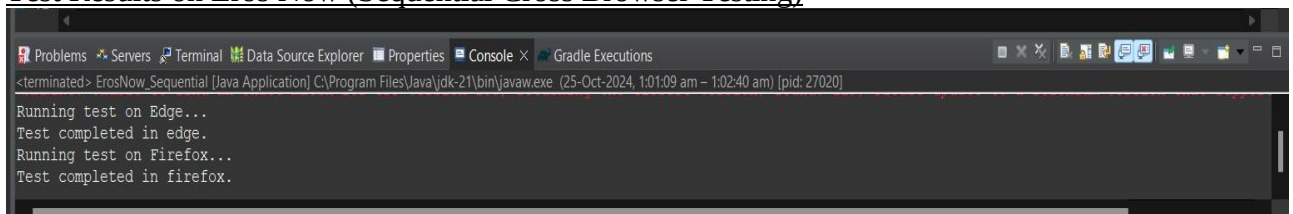
Test Results on Disney+ Hotstar (Sequential & Parallel Cross Browser Testing)



```
<terminated> DisneyHotstar_Sequential [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (25-Oct-2024, 12:48:21 am – 12:49:51 am) [pid: 37524]
Running test on Edge...
Test completed in edge.
Running test on Firefox...
Test completed in firefox.
```

Fig-5

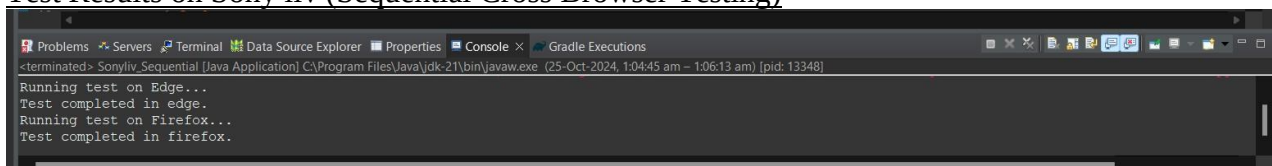
Test Results on Eros Now (Sequential Cross Browser Testing)



```
<terminated> ErosNow_Sequential [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (25-Oct-2024, 1:01:09 am – 1:02:40 am) [pid: 27020]
Running test on Edge...
Test completed in edge.
Running test on Firefox...
Test completed in firefox.
```

Fig-6

Test Results on Sony liv (Sequential Cross Browser Testing)



```
<terminated> Sonyliv_Sequential [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (25-Oct-2024, 1:04:45 am – 1:06:13 am) [pid: 13348]
Running test on Edge...
Test completed in edge.
Running test on Firefox...
Test completed in firefox.
```

Fig-7